GEP v1.1

Phase 3 개발계획서

레벨 1~2 완성: 게스트 제한 + OAuth + 서버 저장 + 통계 UI

문서번호	GEP_039
작성일	2026.02.23
작성자	노팀장 (Claude Opus 4.6)
수신	개발관리 팀장004 → 고팀장(클로드 코드)
승인자	조대표님
기준 문서	GEP_038 (서비스레벨별 통합 서비스 시나리오 정의서)
상태	조대표님 승인 대기

1. Phase 3 개요

Phase 2에서 구축한 Supabase 인프라(8개 테이블, Feature Gate, authStore, statsService)를 실제 서비스로 활성화하는 단계입니다. Phase 3 완료 시 게스트는 360문제 맛보기 + 기본 통계를 경험하고, 회원(레벨2)은 전체 1,080문제 + 서버 기반 상세 통계 + 틀린문제 모아풀기를 사용할 수 있습니다.

1.1 조대표님 확정 사항

- OAuth 제공자: Google 단독 (Kakao 는 검증 후 추가)
- 레벨 2 활성화: 베타 테스터 수동부여 (관리자가 DB 에서 service_level 변경)
- 개발 단위: 중단위 4 단계 (STEP 별 완료 보고 후 다음 STEP 진행)

2. 기존 인프라 현황 (Phase 2 완료)

아래 파일들은 Phase 2에서 이미 구축 완료되었습니다. Phase 3에서는 이 인프라를 활성화하고 연동합니다. 기존 코드 수정은 최소화하고, 신규 파일 추가 방식(레고블럭)으로 진행합니다.

파일	구분	상태
supabase/schema.sql	DB 스키마 (8개 테이블)	완료 — 수정 불필요
src/lib/supabase.js	Supabase 클라이언트	완료 — 수정 불필요
src/config/featureFlags.js	Feature Gate 중앙관리	완료 — 수정 불필요
src/services/statsService.js	게스트/회원 분기 어댑터	완료 — STEP 3에서 활성화
src/stores/authStore.js	인증 상태 관리	완료 — STEP 2에서 연동
src/pages/Home.jsx	홈 화면	TODO 주석 있음 — STEP 1~2에서 수정

src/pages/Question.jsx	문제 풀이	async + recordAttempt 경유 — STEP 1에서 제한 로직 추가

3. Phase 3 완료 후 레벨별 기능 비교

GEP_038 기준. Phase 3 완료 시 레벨 1과 레벨 2의 차이:

항목	구분	레벨1 (게스트)	레벨2 (회원)
문제 범위		세부과목별 30문제 (총 360)	전체 1,080문제
인증		불필요	Google OAuth
데이터 저장		localStorage만	localStorage + Supabase
통계	누적/금일	O (로컬)	O (서버)
통계	과목별 정답률	O (전체 공개)	O (서버 기반 상세)
통계	세부과목별 상세	X	O
통계	틀린문제 카운트	X	O
틀린문제풀기		X	O
이어풀기		O (로컬)	O (서버 싱크)

4. STEP 별 실행 계획

순서 변경 불가. 각 STEP 완료 후 개발관리 팀장이 검증하고, 최상위 기획창에 보고합니다.

STEP	명칭	핵심 작업	선행조건	완료기준
1	**게스트 문제 제한**	세부과목별 30문제 제한 + 소진 시 유도 팝업 + 레벨1 로컬 통계(누적/금일/과목별 정답률)	없음	360문제 소진 후 차단 확인
2	**Google OAuth 로그인**	Supabase Auth + Google OAuth 설정 + 로그인/로그아웃 UI + 회원 전환 흐름 + 베타 레벨2 수동부여	STEP 1 완료	Google 로그인 성공 + DB users 행 생성
3	**서버 저장 + 틀린문제 풀기**	레벨2 회원 attempts 서버 저장 활성화 + 이어풀기 Issue-3 정상화 + 틀린문제 모아풀기 기능	STEP 2 완료	서버 attempts 기록 확인 + 틀린문제 리스트 정상
4	**통계 UI + 통합 테스트**	레벨1 로컬 통계 UI + 레벨2 서버 통계 UI(세부과목별 상세 + 틀린문제 카운트) + 전체 통합 테스트 + Vercel 배포	STEP 3 완료	게스트/회원 시나리오 전수 통과 + 배포 완료

4.1 STEP 1: 게스트 문제 제한 + 레벨 1 로컬 통계

목표: 현재 무제한인 게스트 문제풀이를 세부과목별 30문제로 제한하고, 기본 통계를 제공합니다.

작업 내용

- 문제 제한 로직: 세부과목 선택 시 최대 30 문제만 로드. 30 문제 소진 시 다음 문제 차단
- 유도 팝업: 30 문제 소진 시 "전체 1,080 문제를 풀려면 회원가입하세요" 팝업 표시. 팝업에서 로그인 화면으로 이동 또는 다른 세부과목 선택 가능
- 레벨 1 통계 (GEP_038 확정): 누적 풀이수/정답수, 금일 풀이수/정답수, 과목별(3 과목) 정답률. 모두 로컬 기반, 전체 공개
- Home.jsx 수정: 세부과목 카드에 "N/30" 진행률 표시

수정 파일 (예상)

- src/pages/Question.jsx — 30 문제 제한 로직 추가
- src/pages/Home.jsx — 진행률 표시 + 소진 상태 UI
- src/components/LimitPopup.jsx — 신규 생성 (유도 팝업)
- src/components/StatsPanel.jsx — 신규 생성 (레벨 1 기본 통계)

완료 기준

- 세부과목 30 문제 소진 후 31 번째 문제 진입 차단 확인
- 유도 팝업 정상 표시 + 다른 과목 이동 가능 확인
- 레벨 1 통계(누적/금일/과목별) 정상 표시 확인
- npm run build 성공 + 기존 기능 회귀 테스트 통과

4.2 STEP 2: Google OAuth 로그인

목표: Supabase Auth + Google OAuth를 연동하여 게스트 → 회원 전환 흐름을 완성합니다.

작업 내용

- Supabase 대시보드: Google OAuth Provider 활성화 (Google Cloud Console 에서 Client ID/Secret 발급 필요 — 조대표님 직접 설정 또는 가이드 제공)
- 로그인 UI: Home.jsx 의 TODO 위치에 "Google 로 시작하기" 버튼 배치. 모바일 최적화
- 로그인 흐름: 버튼 클릭 → Supabase signInWithOAuth({provider: 'google'}) → 콜백 → authStore 업데이트 → users 테이블 행 생성(service_level=1)
- 로그아웃: 설정 화면에 로그아웃 버튼. signOut() → authStore 리셋 → 게스트로 복귀
- 베타 레벨 2 부여: 관리자(조대표님)가 Supabase 대시보드에서 users.service_level = 2 로 직접 변경. 앱에서 다음 로그인 시 자동 반영
- Feature Gate 연동: 로그인 시 service_level 조회 → authStore.userFeatures 갱신 → UI 자동 분기

수정/생성 파일 (예상)

- src/stores/authStore.js — signIn/signOut/onAuthStateChange 로직 추가
- src/pages/Home.jsx — 로그인 버튼 + 회원 상태 표시
- src/components/LoginButton.jsx — 신규 생성
- src/pages/Settings.jsx — 로그아웃 + 구독 정보 표시 (신규 또는 수정)
- .env.local — Google OAuth 관련 환경변수 (이미 Supabase URL/KEY 존재)

완료 기준

- Google 로그인 버튼 클릭 → Google 인증 → 앱 복귀 → 회원 상태 표시
- Supabase users 테이블에 새 행 생성 확인
- 로그아웃 → 게스트 상태로 복귀 확인
- service_level=2 수동 변경 후 앱에서 레벨 2 기능 활성화 확인

4.3 STEP 3: 서버 저장 + 틀린문제 모아풀기

목표: 레벨2 회원의 학습 데이터를 Supabase에 저장하고, 틀린문제 모아풀기 기능을 활성화합니다.

작업 내용

- attempts 서버 저장: statsService.recordAttempt()에서 레벨 2 이상일 때 Supabase attempts INSERT 활성화 (코드는 Phase 2 에서 이미 작성됨, 게이트만 열기)
- question_stats 집계: attempts INSERT 시 동시에 question_stats UPSERT (정답률 캐시). 통계 조회 시 question_stats 만 읽어 성능 확보
- 이어풀기 Issue-3 정상화: 현재 이어풀기 기능의 알려진 버그 수정. 서버 progress 테이블 연동 (레벨 2)
- 틀린문제 모아풀기: attempts 데이터에서 is_correct=false 인 문제 목록 조회 → 별도 학습 모드로 제공. study_mode='wrong_review'로 기록
- 오프라인 방어: 네트워크 실패 시 로컬에만 저장 + 재접속 시 pending queue 처리 (Phase 2 설계 활용)

수정/생성 파일 (예상)

- src/services/statsService.js — 서버 저장 게이트 활성화 + question_stats UPSERT
- src/pages/WrongReview.jsx — 신규 생성 (틀린문제 모아풀기 화면)
- src/pages/Home.jsx — 틀린문제풀기 진입점 추가 (레벨 2 전용)
- src/stores/progressStore.js — 이어풀기 버그 수정 (Issue-3)

완료 기준

- 레벨 2 회원으로 문제 풀기 → Supabase attempts 테이블에 행 생성 확인
- question_stats 에 집계 반영 확인
- 틀린문제 목록 정상 조회 + 모아풀기 진입 확인

- 이어풀기: 앱 종료 후 재접속 시 마지막 위치에서 이어서 풀기 확인
- 게스트 상태에서 서버 저장 시도 없음(Supabase INSERT skip) 확인

4.4 STEP 4: 통계 UI + 통합 테스트 + 배포

목표: 레벨별 통계 화면을 구현하고, 전체 시나리오를 검증한 후 Vercel에 배포합니다.

작업 내용

- 레벨 1 통계 UI: STEP 1 에서 구현한 StatsPanel 마무리. 누적/금일/과목별 정답률 대시보드
- 레벨 2 통계 UI: 서버 데이터 기반 세부과목별(12 개) 상세 정답률 + 틀린문제 카운트 표시. question_stats 조회
- Feature Gate UI 분기: 게스트 → 레벨 1 통계만 표시 + "더 자세한 통계는 회원 전용" 안내. 회원 레벨 2 → 전체 통계 표시
- 맥락적 유도 (GEP_038 §5.2): 세부과목 30 문제 소진 시 레벨 2 유도 (STEP 1 팝업 활용)

통합 테스트 시나리오

- 시나리오 A (게스트): 앱 진입 → 과목 선택 → 30 문제 풀기 → 소진 팝업 → 다른 과목 이동 → 레벨 1 통계 확인
- 시나리오 B (신규 회원): Google 로그인 → 레벨 1 상태 → 관리자 레벨 2 부여 → 재로그인 → 전체 문제 해금 확인
- 시나리오 C (레벨 2 학습): 문제 풀기 → 서버 저장 확인 → 틀린문제풀기 → 레벨 2 통계 확인 → 이어풀기
- 시나리오 D (오프라인): 네트워크 끊김 → 문제 풀기 → 로컬 저장 → 네트워크 복귀 → 서버 싱크 확인
- 시나리오 E (회귀): Phase 1 기존 기능 전수 테스트 (과목 선택, 문제 풀기, 정답 확인 등)

완료 기준

- 시나리오 A~E 전수 통과
- npm run build 성공
- Vercel 배포 완료 + gepv11.vercel.app 정상 접속
- 모바일(iOS Safari, Android Chrome) 실기기 테스트 통과

5. 보고 체계

각 STEP 완료 시 아래 보고 경로를 따릅니다:

STEP	개발관리 팀장 → 고팀장 지시	고팀장 → 개발관리 팀장 보고	개발관리 팀장 → 최상위 기획창 보고
1	게스트 제한 + 로컬 통계 구현 지시	코드 변경 목록 + 테스트 결과	STEP 1 완료 보고 + 검증 결과

2	Google OAuth 구현 지시	로그인 성공 스크린샷 + DB 확인	STEP 2 완료 보고 + 검증 결과
3	서버 저장 + 틀린문제풀기 지시	attempts 행 확인 + 기능 테스트	STEP 3 완료 보고 + 검증 결과
4	통계 UI + 통합 테스트 지시	전수 시나리오 결과표	Phase 3 최종 완료 보고

6. 개발 원칙 (필수 준수)

- 레고블럭: 기존 파일 수정 최소화, 신규 파일 추가 방식. statsStore.js 수정 금지
- SSOT: attempts 원장 단일화. 통계는 question_stats 캐시에서만 조회
- 모바일 퍼스트: 모든 UI 는 375px 기준 설계. 터치 영역 44px 이상
- 하드코딩 금지: 과목명/세부과목명은 exams.json 에서 읽기. 코드에 직접 입력 금지
- 에러 방어: 네트워크 실패, Supabase 장애 시 앱 중단 없이 로컬 폴백
- 빌드 검증: 매 STEP 완료 시 npm run build 성공 필수

7. 주의사항

Google OAuth 설정: Google Cloud Console에서 OAuth 2.0 Client ID 발급이 필요합니다. Authorized redirect URI는 Supabase 대시보드에서 확인 가능합니다. 이 설정은 조대표님이 직접 수행하거나, 고팀장이 가이드를 제공합니다.

localStorage 키 충돌 방지: Phase 1에서 사용 중인 localStorage 키와 충돌하지 않도록 새 키에는 'gep_v2_' 접두사를 사용합니다.

베타 테스터 운영: 레벨2 부여는 Supabase 대시보드 > Table Editor > users > service_level 컬럼을 2로 변경하는 방식입니다. 초기 10~50명 대상으로 피드백 수집 후 정식 수익화를 검토합니다.

노팀장 (Claude Opus 4.6) | 2026.02.23